

Research Proposal — Kinematic Recovery Perturbation (KRP)

Working title: *KRP: Policy-Aware, Kinematically-Consistent Recovery Perturbations for Camera-Based End-to-End Planners*

Abstract: RAP's recovery perturbations are **manually designed and single-frame teleports**, and are therefore physically unrealizable. We replace them with **kinematically feasible control sequences** and use **gradients**, rather than hand-crafted rules, to select the perturbations — and this gradient path exists only under RAP's **rasterized** observation.

1. Problem: RAP's Recovery Perturbations Are Physically Unrealizable

- RAP's premise: imitation-learning planners fail in **closed loop** because they have never encountered recovery states.
- RAP's response: use rasterization to render off-log recovery states inexpensively.
- **But how are these states generated?** In the released code `process_data/create_openscene_metadata_purturbed.py` (`get_ego_params`, L246–262; per-frame gating, L330–336), only a **single frame**, `batch[3]`, is perturbed within a 14-frame window:

$$(\mathbf{E1}) \quad s_0 \leftarrow s_0 + \xi, \quad \xi \sim \mathcal{U}(\Xi), \quad s_t \text{ unchanged for } t \neq 0$$

$$\Delta x, \Delta y \sim \mathcal{U}(-0.5, 0.5) \text{ m}, \quad \Delta \psi \sim \mathcal{U}(-15^\circ, 15^\circ), \quad \Delta v = \mathcal{U}(-0.2, 0.2) \cdot v$$

Three Consequences

① **Kinematically infeasible.** With `NAVSIM_INTERVAL_LENGTH = 0.5` s, substituting:

$$\omega = \frac{0.26 \text{ rad}}{0.5 \text{ s}} = 0.52 \text{ rad/s} \implies a_{\text{lat}} = v\omega \Big|_{v=20} \approx 10.5 \text{ m/s}^2$$

- This exceeds the tire–road friction limit → **the training state is fundamentally unreachable in closed loop.**
- This is a one-line mechanics calculation, and it sits in a core assumption rather than an implementation detail.

② **It creates a shortcut.** Because the history frames are left unperturbed, the offset can be read directly from the frame-to-frame inconsistency:

$$\hat{\xi} \approx s_0 - \kappa(s_{-1}, a_{-1})$$

- The network learns to "detect the jump and cancel it" instead of "inferring the lane from the images."
- This is a form of copycat behavior / causal confusion (de Haan 2019; Codevilla 2019), except here the shortcut is created by the augmentation itself.

③ **The perturbation vanishes at standstill.** The speed perturbation is multiplicative, so $v = 0 \Rightarrow \Delta v = 0$ — parking / launch-from-rest scenarios (where the inertia problem is especially common) receive no effective perturbation.

Consequences ① and ② are shown together in the **left half of Figure 2(a)**: the three history frames remain unchanged on the lane centerline, while the current frame is moved by a "teleport" arrow to a pose offset by 0.5 m / 15°.

2. The Reference Method, KING, and Why It Cannot Be Transferred Directly

2.1 What is KING?

- KING (ECCV 2022) performs **safety-critical scenario generation**: given a fixed ego policy, it optimizes the behavior of **background vehicles** so that they collide with the ego.
- Its central claim: **do not edit states by hand; edit actions**, and let a differentiable kinematic model generate the states:

$$(E2) \quad s_{t+1} = \kappa(s_t, a_t + w_t)$$

- Benefit: the resulting states are **physically reachable by construction**, and they can be optimized with gradients.
- **This is exactly what RAP's recovery perturbations lack** — scenario generation made the shift from "hand-tuned parameters" to "kinematically constrained optimization" with KING, yet **ego-side recovery perturbation remains stuck in the pre-KING paradigm**.

2.2 KING Has Two Gradient Paths

$$\frac{dC}{d\theta} = \underbrace{\frac{dC}{d\theta} \Big|_{\text{direct}}}_{\text{motion of the background vehicle itself}} + \underbrace{\frac{dC}{d\theta} \Big|_{\text{indirect}}}_{\text{through the ego vehicle's response}}$$

- **Direct path:** $a_t^{i>0} \rightarrow s_{t+1}^{i>0} \rightarrow \mathcal{C}$, namely

$$\frac{dC}{da_t^{i>0}} \Big|_{\text{direct}} = \frac{\partial \mathcal{C}}{\partial s_{t+1}^{i>0}} \cdot \frac{\partial \kappa(s_t^{i>0}, a_t^{i>0})}{\partial a_t^{i>0}}$$

Both factors are available in closed form (derivative of the distance function + derivative of the bicycle model). **The observation o_t never appears.**

- **Indirect path:** $a_t^{i>0} \rightarrow s_{t+1}^{i>0} \rightarrow \boxed{o_{t+1}} \rightarrow \pi(o_{t+1}) \rightarrow s_{t+2}^0 \rightarrow \mathcal{C}$ requires $\partial o / \partial s$ — the CARLA renderer is a black box, so this is **unavailable**.
- **What KING does: simply drop the indirect path.** The remaining gradient is biased but nonzero and points in a useful direction ("move the vehicle toward where the ego will be"), so the method still works.

- The enabling condition: the cost $\mathcal{C}$ is geometric (collision distance), so the direct path reaches the cost without ever touching the observation.

2.3 But That Assumption Does Not Hold for Ego Recovery

- We define the recovery cost as a function of the planner's **output trajectory** τ :

$$\mathcal{L}_{\text{rec}} = \mathcal{L}(\tau), \quad \tau = \pi_{\theta}(o(s^0(w)))$$

- Under this definition, w can affect $\mathcal{L}_{\text{rec}}$ **only** through o , therefore

$$\left. \frac{d\mathcal{L}_{\text{rec}}}{dw} \right|_{\text{direct}} \equiv 0 \quad (\text{exactly})$$

- Compare with KING: its cost $\mathcal{C}(s)$ is geometric, so the direct path reaches it without touching the observation; our cost is a function of **policy behavior**, making the **observation path the only path**.
- KING can afford to remove the observation path; we cannot — once it is removed, nothing remains.

2.4 Figure 1

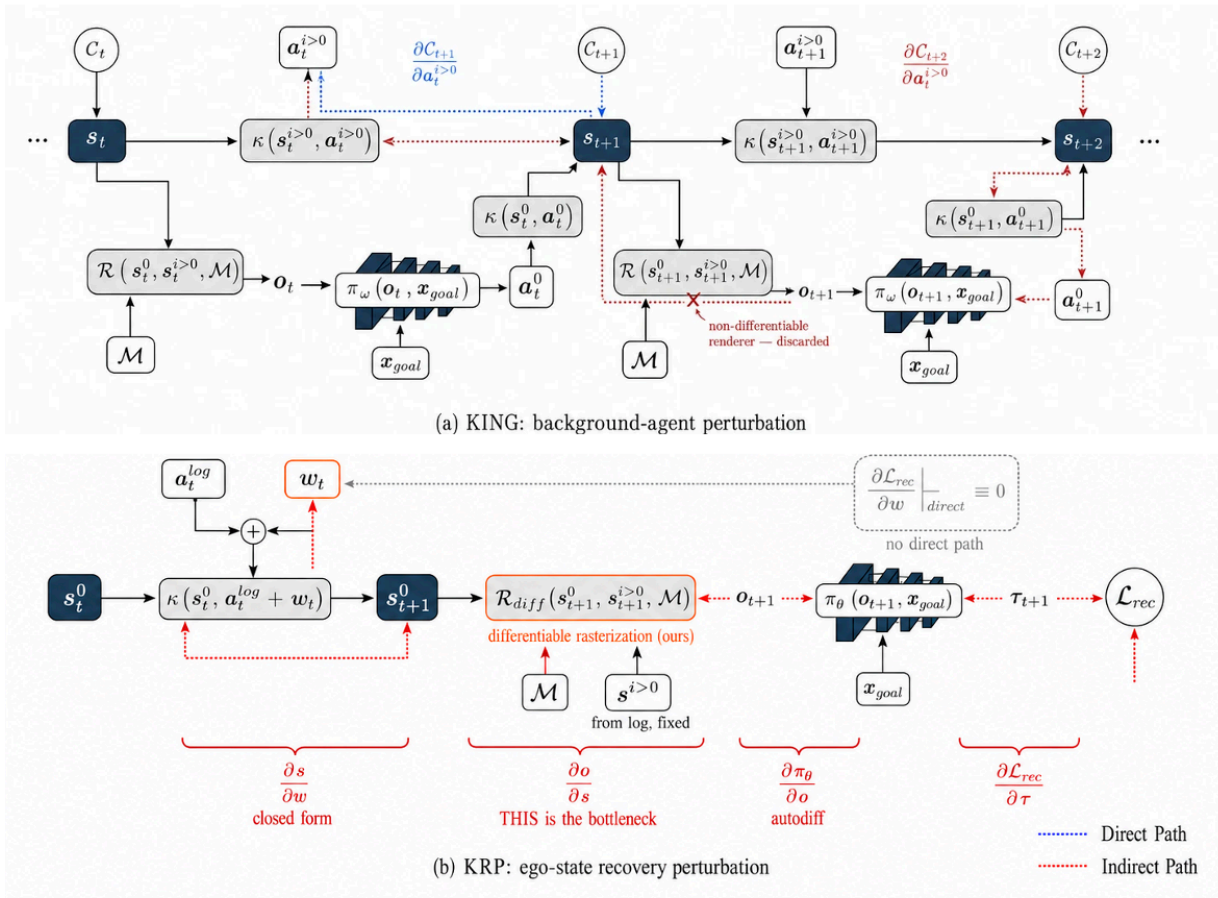


Figure 1 | (a) KING: the cost $\mathcal{C}$ is geometric, so the blue direct path $a_t^{i>0} \rightarrow \kappa \rightarrow s_{t+1} \rightarrow \mathcal{C}_{t+1}$ bypasses the renderer and survives; the red indirect path is cut at the renderer and discarded, yet the method still works. (b) KRP: there is no direct path (gray box at the upper right, $\equiv 0$). The only red path

$\mathcal{L}_{\text{rec}} \rightarrow \tau \rightarrow \pi_\theta \rightarrow o \rightarrow \mathcal{R}_{\text{diff}} \rightarrow s^0 \rightarrow \kappa \rightarrow w_t$ **must pass through the renderer**. Of the four Jacobian factors along this chain, $\partial\pi_\theta/\partial o$ is supplied by automatic differentiation and $\partial s/\partial w$ is closed form; the sole bottleneck is $\partial o/\partial s$ — and **differentiable rasterization is precisely what makes it exist**.

3. Method: KRP

3.1 Side-by-Side Comparison with the Two Prior Works

	Optimization variable	Constraint	Cost	Selection
RAP	State offset ξ	none	none	$\xi \sim \mathcal{U}(\Xi)$, hand-fixed
KING	Background action θ	$s_{t+1}^{i>0} = \kappa(\cdot)$	geometric (collision distance)	$\nabla_\theta \mathcal{C}$
KRP (ours)	Ego disturbance $w_{-K:0}$	$s_{t+1}^0 = \kappa(s_t^0, a_t^{\text{log}} + w_t)$	perception-driven	$\nabla_w \mathcal{L}_{\text{rec}}$

- First two columns: KRP inherits KING's **parametrization**.
- Third column: this is where the difficulty is — the cost changes from geometric to perception-driven, so we must compute the factor below.

3.2 The Gradient We Must Compute

$$\frac{\partial \mathcal{L}_{\text{rec}}}{\partial w} = \sum_t \underbrace{\frac{\partial \mathcal{L}_{\text{rec}}}{\partial \tau}}_{\text{(i)}} \underbrace{\frac{\partial \pi_\theta}{\partial o_t}}_{\text{(ii) autodiff}} \underbrace{\frac{\partial o_t}{\partial s_t}}_{\text{(iii) observation model}} \underbrace{\frac{\partial s_t}{\partial w}}_{\text{(iv) bicycle model: closed form}}$$

- These four factors **correspond one-to-one to the four red braces at the bottom of Figure 1(b)**.
- (i) and (ii) are supplied by automatic differentiation; (iv) is closed form. **Only (iii) is the bottleneck**.

3.3 The Bidirectional Lock: Why It Must Be RAP's Rasterization

Observation source	$\partial o / \partial s$	Why
Logged real video	does not exist	the vehicle never occupied that pose, so the image does not exist — not "hard to compute" but "no object to compute"
CARLA / game engine	non-differentiable	black box; precisely why KING was forced to drop the indirect path
NeRF / 3DGS	learned, degrades under extrapolation, 10^2 – $10^3 \times$ slower	and the perturbed state lives exactly in the extrapolation regime ; unusable inside an inner loop
Rasterized primitives (RAP)	analytic, exact, cheap	the scene is a known geometric object; $s \mapsto o$ is the closed chain $T(\delta) \rightarrow \text{projection} \rightarrow (\text{soft}) \text{ rasterization}$

KING on the ego $\implies$ needs $\frac{\partial o}{\partial s} \implies$ needs rasterized observation

RAP recovery augmentation $\implies$ needs physical realizability $\implies$ needs kinematic parametriza

- Neither direction is rhetorical: the former is tested by Experiment 3, the latter by Experiment 1.

3.4 The Optimization Problem

$$(\mathbf{E3}) \quad w_{-K:0}^* = \arg \min_w \|w\|_W \quad \text{s.t.} \quad \begin{cases} s_{t+1} = \kappa(s_t, a_t^{\log} + w_t) & \text{feasible by construction} \\ \mathcal{L}_{\text{rec}}(\pi_\theta, o(s_{-K:T})) > \varepsilon & \text{learner fails} \\ w_t \in \mathcal{W}(v_t) & \text{actuation limits} \end{cases}$$

- Intuition: **find the smallest physically feasible perturbation that the current planner cannot recover from.**
- Note the objective is **minimum-norm**: not "make the planner fail as much as possible" (that would be gradient ascent), but "the smallest perturbation that just crosses the failure threshold ε ."
- The last constraint comes from non-holonomy: $\omega = \frac{v \tan \delta_{\text{steer}}}{L} \implies v = 0 \Rightarrow \omega = 0 \rightarrow$ **heading cannot be perturbed at standstill**; the current implementation violates this, whereas our formulation enforces it, which is why the perturbation magnitude necessarily becomes **speed-dependent**.

3.5 Two Code-Level Modifications

- **M1 | Perturbation parametrization** — replace "state rewriting" with "control-space disturbance rolled forward through κ ," covering the **entire history window**. $\rightarrow$ Corresponds to the **right half of Figure 2(a)** (same start, same terminal, but a continuous trajectory throughout) and the κ node in Figure 2(b).
- **M2 | Differentiable rendering** — replace the rasterizer with a **differentiable (soft) rasterizer**, primitives unchanged. $\rightarrow$ Corresponds to $\mathcal{R}_{\text{diff}}$ in Figure 2(b), i.e. the sole bottleneck factor $\partial o / \partial s$ from §3.2.

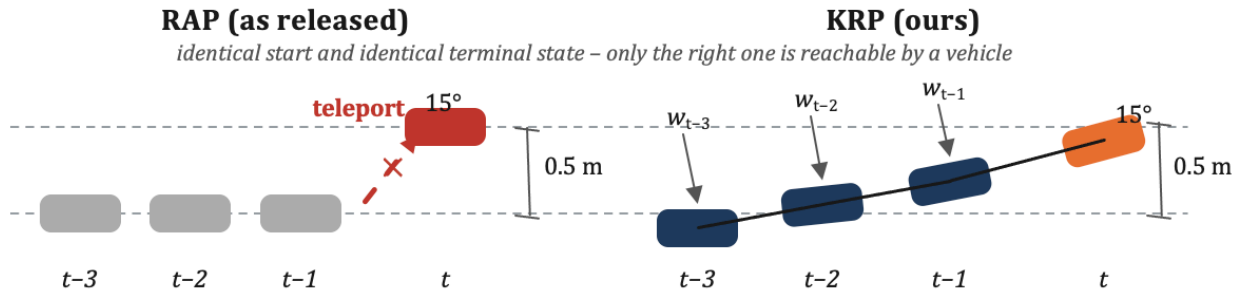
Together these two modifications make the entire chain from w to $\mathcal{L}_{\text{rec}}$ **fully analytically differentiable**, so (E3) can be solved with a first-order method.

3.6 The Mining Loop and Its Use in Training

- **Mining:** for each scene, start from $w = 0$ and iterate along $\partial \mathcal{L}_{\text{rec}} / \partial w$ until $\mathcal{L}_{\text{rec}} > \varepsilon$ is satisfied with $\|w\|_W$ minimized, yielding w^* .
- **Training:** w^* defines a perturbed clip $\rightarrow$ re-render across **7 cameras $\times$ K frames** $\rightarrow$ obtain a recovery target from a **privileged expert** $\rightarrow$ mix into the training set under a **fixed budget**.
- The fixed budget is necessary: all baselines (B0–B4) must consume the same number of augmented samples, otherwise the RQ2 comparison does not hold.

3.7 Figure 2

(a) The same target offset, reached two ways



history frames unperturbed $\Rightarrow \hat{\xi} \approx s_0 - \kappa(s_{-1}, a_{-1})$ is directly readable (shortcut)

$$\omega = 0.26 \text{ rad}/0.5 \text{ s} = 0.52 \text{ rad/s}$$

$$\Rightarrow a_{\text{lat}} = v\omega|_{v=20} \approx 10.5 \text{ m/s}^2$$

beyond friction limit – unreachable

$$s_{k+1} = \kappa(s_k, a_k^{\text{log}} + w_k)$$

feasible by construction; history is re-rendered consistently

(b) Gradient-based mining of w^* , and its use in training

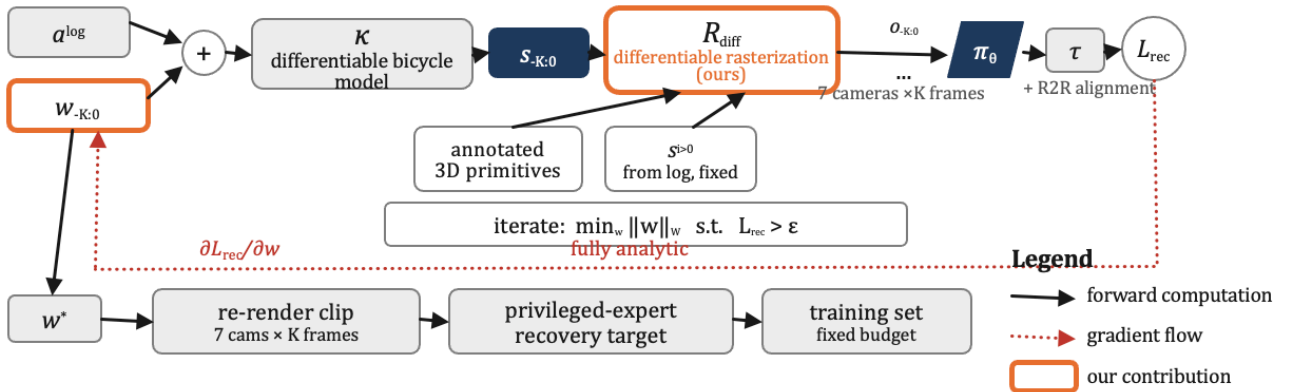


Figure 2 | (a) Perturbation geometry. Left: RAP as released — the three history frames stay unchanged on the lane centerline, while the current frame is teleported to the terminal; hence ① the state is kinematically unreachable ($a_{\text{lat}} \approx 10.5 \text{ m/s}^2$), and ② the offset can be read directly from the frame-to-frame inconsistency $\hat{\xi} \approx s_0 - \kappa(s_{-1}, a_{-1})$, forming a shortcut. Right: KRP — **the start and terminal are identical to the left**, but the entire trajectory evolves via $s_{k+1} = \kappa(s_k, a_k^{\text{log}} + w_k)$, hence feasible by construction, and the history frames are re-rendered consistently. **The same target offset, but only the right path is one a vehicle can drive.**

(b) Mining loop and use in training. The disturbance $w_{-K:0}$ is added to the logged action a^{log} and passed through κ to obtain the state sequence, which $\mathcal{R}_{\text{diff}}$ (together with annotated 3D primitives and the background-vehicle states $s^{i>0}$ taken from the log and held fixed) renders into 7-camera $\times$ K -frame observations, fed to the planner π_θ to produce the trajectory τ and the recovery cost $\mathcal{L}_{\text{rec}}$. The red dotted line is the fully analytic gradient return $\partial \mathcal{L}_{\text{rec}} / \partial w$; it iteratively solves $\min_w \|w\|_W$ s.t. $\mathcal{L}_{\text{rec}} > \varepsilon$. The resulting w^* is used to re-render the clip, paired with a privileged-expert recovery target, and merged into the training set under a fixed budget. **The two orange-highlighted items (w 's control-space parametrization and $\mathcal{R}_{\text{diff}}$) are M1 and M2.**